

Recursion in Modern Programming Problems

Zachery Thompson

CS 505: Discrete Structures

Southern New Hampshire University

Dr. Jeffrey Zheng

June 12, 2026

Recursion in Modern Programming Problems

Introduction

Recursion is a programming technique in which a function solves a problem by calling itself on smaller instances of the same problem. Instead of tackling a large problem all at once, recursion breaks it down into smaller pieces until it reaches a simple stopping point known as the base case. Recursive algorithms are often used when working with hierarchical or interconnected data structures such as trees and graphs (Njeru, 2025).

One area where recursion can be useful is tabletop role-playing game (TTRPG) software and virtual tabletop platforms. These systems often represent dungeons as collections of connected rooms that players can explore. As a dungeon grows larger and more complex, the software must determine which rooms can be reached and ensure that all accessible areas are explored. Recursion provides a practical way to solve this problem through graph traversal algorithms such as depth-first search (DFS). Modern programming languages, including Python, Java, and C++, all support recursion, making it a valuable tool for game developers and software engineers.

A Real-World Programming Problem: Exploring a Dungeon Map

Dungeon exploration is a common feature in both video games and digital TTRPG tools. A dungeon is usually made up of multiple rooms connected by doors, hallways, tunnels, or other pathways. In software, these dungeons can be represented as graphs, where rooms are modeled as vertices (nodes) and the pathways between them are modeled as edges.

For example, a dungeon may contain a starting room that connects to several nearby rooms, each of which may lead to even more areas. A game engine might need to reveal unexplored rooms, determine which locations are reachable, verify that all rooms are accessible, or generate a complete map for players. As the number of rooms increases, keeping track of visited locations becomes more difficult.

Another complication occurs when a dungeon contains loops. A player might move from Room A to Room B, then to Room C, and eventually return to Room A through a different route. If the program does not track visited rooms correctly, it could repeatedly explore the same locations and potentially enter an infinite loop. Because of this, the software needs a reliable way to explore connected rooms while tracking which of them have already been visited.

This type of problem is well-suited to recursive graph traversal algorithms because a dungeon's structure closely resembles the graphs studied in discrete mathematics. Devansh (2022) notes that recursion is especially useful for traversing graphs and tree-like structures because it naturally follows connected paths while reducing a larger problem into smaller steps.

Using Recursion to Solve the Problem

One way to solve dungeon exploration is by using a recursive depth-first search algorithm. The algorithm starts in a room and marks it as visited. It then examines each neighboring room connected to the current location. Whenever it finds an unvisited room, it calls itself to explore that room before returning to continue checking the remaining connections.

The recursive process relies on two important parts: a base case and a recursive case. The base case occurs when there are no unvisited neighboring rooms left to explore. At that point, the

function returns to the previous call and continues searching other paths. The recursive case occurs whenever an unvisited room is found, causing the function to call itself again using that room as the new starting point. As Mayo (2024) explains, effective recursion requires both a stopping condition and an action that moves the program closer to reaching that condition.

The logic can be represented with the following pseudocode:

```
Explore(Room)
    Mark Room as Visited
    For Each Connected Room
        If Connected Room Not Visited
            Explore(Connected Room)
```

This approach allows the program to visit every reachable room while avoiding duplicate visits. It also closely mirrors how a player might mentally explore a dungeon, moving deeper into unexplored areas and backtracking when a path dead-ends.

One advantage of recursion is that the code often reflects the structure of the problem more naturally than an iterative solution. Recursive solutions are usually shorter and easier to read when working with graphs and trees. Recursion also handles backtracking naturally because the call stack keeps track of previously visited locations and returns to them automatically when needed.

Despite these advantages, recursion does have drawbacks. Each recursive call uses additional memory on the call stack. If a dungeon becomes extremely large, the number of recursive calls could exceed the available stack space and result in a stack overflow error. In

those situations, an iterative solution using an explicit stack may be more efficient. Even with these limitations, recursion remains a practical and elegant solution for many dungeon exploration problems (Dale, 2018).

Modern Programming Languages and Recursive Problem-Solving

Most modern programming languages support recursion. Python, Java, and C++ all allow functions or methods to call themselves, making recursive solutions relatively easy to implement.

Python is often used to teach recursion because its syntax is clean and easy to read. Developers can implement recursive algorithms with very little boilerplate code. This simplicity makes Python a popular choice for demonstrating graph traversal algorithms such as depth-first search. However, Python also limits recursion depth to help prevent excessive memory usage and stack overflow errors.

Java also supports recursive methods and is widely used in education and professional software development. Its object-oriented design allows programmers to model dungeon rooms and their connections as objects, making recursive traversal algorithms easy to organize and maintain.

C++ supports recursion while providing greater control over memory and performance. Because of its speed and efficiency, C++ is commonly used in game development and other performance-critical applications. Recursive graph traversal algorithms written in C++ can efficiently explore large and complex dungeon maps.

Although the syntax differs between these languages, the underlying concept remains the same. Each language uses a call stack to keep track of active function calls. This allows the program to move deeper into the dungeon and then return to previous rooms as the search continues. Njeru (2025) explains that recursive calls create a chain of function executions that remain on the call stack until the base case is reached, after which the stack unwinds and returns control through each previous function call.

Conclusion

Recursion is a powerful programming technique that solves large problems by breaking them into smaller, more manageable pieces. A practical example can be found in TTRPG dungeon exploration, where software must navigate networks of connected rooms. By modeling a dungeon as a graph and applying a recursive depth-first search algorithm, a game engine can systematically explore every reachable location while avoiding duplicate visits and infinite loops.

Modern programming languages such as Python, Java, and C++ all provide strong support for recursive programming. Although recursion can raise memory concerns when dealing with extremely large problems, it often yields concise, readable solutions well-suited to hierarchical data structures. For these reasons, recursion remains an important technique in software development, algorithm design, and game programming.

References

Dale, W. (2018, May 9). Recursion: The pros and cons. Medium.

<https://medium.com/@williambdale/recursion-the-pros-and-cons-76d32d75973a>

Devansh. (2022, March 19). How to get good at recursion: For your FAANG interviews, work, or competitive coding. Geek Culture. [https://medium.com/geekculture/how-to-get-good-](https://medium.com/geekculture/how-to-get-good-at-recursion-377a64ed65bc)

[at-recursion-377a64ed65bc](https://medium.com/geekculture/how-to-get-good-at-recursion-377a64ed65bc)

Mayo, P. (2024, September 23). What is recursion? An introduction and technical discussion.

Medium. [https://medium.com/launch-school/what-is-recursion-an-introduction-and-](https://medium.com/launch-school/what-is-recursion-an-introduction-and-technical-discussion-3c81547d03f0)

[technical-discussion-3c81547d03f0](https://medium.com/launch-school/what-is-recursion-an-introduction-and-technical-discussion-3c81547d03f0)

Njeru, J. W. M. (2025, December 17). Definitive guide to recursive programming. Medium.

[https://medium.com/@Munene254_/definitive-guide-to-recursive-programming-](https://medium.com/@Munene254_/definitive-guide-to-recursive-programming-2a28ca4f3ec5)

[2a28ca4f3ec5](https://medium.com/@Munene254_/definitive-guide-to-recursive-programming-2a28ca4f3ec5)